Twitter Sentiment Analysis Using Machine Learning

A Comparative Study of Classification Algorithms for Polarity Detection

Aatazaz Javed
aatazaz.j27@gmail.com

MS Data Science
IM Sciences Peshawar

April 18, 2026

Abstract

This research implements a comprehensive sentiment classification system for Twitter posts using multiple supervised learning algorithms. The objective is to automatically categorize tweets as either positive or negative in sentiment. The Sentiment140 dataset, comprising 1.6 million labeled tweets, serves as the foundation for training and evaluation. The complete data mining workflow encompasses text preprocessing (lowercasing, noise removal, stopword elimination, and stemming), feature extraction via TF-IDF vectorization, and model development using eight distinct classifiers including Logistic Regression, Multinomial Naive Bayes, Random Forest, Gradient Boosting, Linear SVM, and an Ensemble Voting mechanism (10). Hyperparameter optimization through grid search with 5-fold cross-validation was conducted to maximize model performance. Experimental results indicate that the tuned Logistic Regression model achieves the highest accuracy of 75.77% with an ROC-AUC score of 0.834. This paper provides a detailed account of the methodology, empirical findings, and visual representations of the results.

Contents

1 Introduction **4**
1.1 Research Objectives 4

2 Dataset **4**
2.1 Source Attribution 4
2.2 Dataset Structure 4
2.3 Data Characteristics 5

3 Exploratory Data Analysis **5**
3.1 Class Distribution Analysis 5
3.2 Text Length Statistics 5
3.3 Key Observational Insights 6

4 Data Mining Methodology **6**
4.1 Text Preprocessing Pipeline 6
4.2 Feature Engineering 6
4.2.1 TF-IDF Vectorization 6
4.3 Model Development 7
4.3.1 Hyperparameter Optimization 7

5 Experimental Results **7**
5.1 Evaluation Metrics 7
5.2 Model Performance Comparison 8
5.3 Best Model Classification Report 8
5.4 Confusion Matrix Analysis 8
5.5 Top Predictive Features 9
5.6 ROC Curve Analysis 9

6 Discussion **10**
6.1 Key Findings 10
6.2 Limitations and Challenges 10
6.3 Future Research Directions 10

7 Conclusion **11**

A Appendix A: Sample Predictions **13**

B Appendix B: System Requirements **13**

C Appendix C: Code Accessibility **13**

1 Introduction

Social media platforms, particularly Twitter (now rebranded as X), generate enormous volumes of user-generated content on a daily basis. Sentiment analysis, also referred to as opinion mining, involves the computational extraction of subjective information from textual data (4). This field has gained significant attention due to its diverse practical applications, including brand reputation monitoring, political sentiment tracking, stock market prediction, and customer feedback analysis (3).

The primary challenge addressed in this project is the development of a binary classification system capable of automatically determining whether a tweet conveys positive or negative sentiment. Several factors complicate this task: the informal nature of social media language, widespread use of slang and abbreviations, the presence of emoticons, and the 280-character limitation imposed by the platform (2).

1.1 Research Objectives

The specific objectives of this study are as follows:

- Acquire and preprocess a large-scale collection of labeled tweets
- Implement various text normalization and cleaning procedures
- Extract numerical feature representations using TF-IDF vectorization
- Train and evaluate multiple machine learning classifiers
- Compare model performance using appropriate evaluation metrics
- Optimize the best-performing model through hyperparameter tuning
- Deploy an interactive system for real-time sentiment prediction on new tweets

2 Dataset

2.1 Source Attribution

The dataset employed in this research is Sentiment140, which contains 1.6 million tweets annotated with sentiment labels. The original work is attributed to **(author?)** (1), who developed the dataset using distant supervision techniques. The dataset is publicly accessible through the Kaggle platform at https://www.kaggle.com/datasets/kazanova/sentiment140.

2.2 Dataset Structure

The Sentiment140 dataset comprises 1.6 million records with the structure detailed in Table 1.

Table 1: Dataset Column Specifications

Column Name	Description
sentiment	Polarity label: 0 indicates negative sentiment, 4 indicates positive sentiment
id	Unique numerical identifier assigned to each tweet
date	Timestamp indicating when the tweet was posted
query	Search query used during collection (consistently "NO_QUERY")
user	Username of the account that published the tweet
text	The complete textual content of the tweet

2.3 Data Characteristics

The dataset exhibits the following properties:

- **Total instances:** 1,600,000 tweets
- **Class distribution:** 800,000 positive (50%), 800,000 negative (50%)
- **Class balance:** Perfectly balanced, making it ideal for binary classification without bias concerns
- **Data completeness:** No missing values present across any columns

3 Exploratory Data Analysis

3.1 Class Distribution Analysis

The dataset is perfectly balanced with exactly 50% positive and 50% negative instances, as shown in Table 2.

Table 2: Sentiment Class Distribution

Sentiment Class	Count	Percentage
Negative (0)	800,000	50%
Positive (4)	800,000	50%
Total	**1,600,000**	**100%**

3.2 Text Length Statistics

Analysis of tweet lengths reveals interesting patterns in user behavior across sentiment classes.

Table 3: Text Length Statistics (measured in characters)

Sentiment	Mean	Median	Std Dev	Minimum	Maximum
Negative	98.5	95.0	42.3	1	280
Positive	89.2	86.0	39.1	1	280

3.3 Key Observational Insights

Based on the exploratory analysis, the following observations can be made:

- Negative tweets tend to be slightly longer on average compared to positive tweets
- Both sentiment classes exhibit similar length distribution patterns
- No significant outliers are present in terms of text length

4 Data Mining Methodology

4.1 Text Preprocessing Pipeline

Effective text preprocessing is essential for sentiment analysis tasks. The following sequence of cleaning operations was applied to each tweet:

1. **Lowercasing:** Converting all text to lowercase to ensure case consistency
2. **URL removal:** Eliminating HTTP/HTTPS links using regular expression patterns
3. **Mention removal:** Removing @username references as they provide limited sentiment information
4. **Hashtag processing:** Removing the # symbol while preserving the accompanying word
5. **Punctuation removal:** Eliminating all punctuation characters
6. **Number removal:** Removing all numeric digits
7. **Stopword removal:** Eliminating common English stopwords (e.g., "the", "and", "is") using the NLTK stopwords corpus
8. **Stemming:** Reducing words to their root forms using the Porter Stemmer algorithm
9. **Whitespace normalization:** Removing extra spaces and trimming the text

4.2 Feature Engineering

4.2.1 TF-IDF Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) was employed to convert the cleaned text into numerical feature vectors. Term Frequency-Inverse Document Frequency (TF-IDF) is used to convert raw text data into numerical vectors, a crucial step for machine learning analysis. Unlike simple word counting, TF-IDF effectively highlights important words while reducing the impact of common, less informative words (such as "the," "is," or "and")

Additionally, character-level n-gram features were extracted using a CountVectorizer with `ngram_range=(2,5)` and `analyzer='char_wb'`, capturing 3,000 character-based features. These were combined with the TF-IDF features to create an 8,000-dimensional feature space.

4.3 Model Development

Eight different machine learning algorithms were implemented and evaluated, as summarized in Table 4.

Table 4: Classification Models Evaluated

Model	Description
Logistic Regression	Linear classifier with L2 regularization
Multinomial Naive Bayes	Probabilistic classifier based on Bayes' theorem with multinomial event model
Random Forest	Ensemble of 200 decision trees with max depth of 50
Gradient Boosting	Sequential ensemble of weak learners with learning rate 0.1
Linear SVM	Support vector machine utilizing a linear kernel
Logistic Regression (Char)	Logistic regression using character-level n-gram features only
Logistic Regression (Combined)	Logistic regression using both word and character features
Ensemble Voting	Soft voting classifier combining Logistic Regression, Naive Bayes, and SVM

4.3.1 Hyperparameter Optimization

Grid search with 5-fold cross-validation was performed to optimize the Logistic Regression model. The parameter grid explored is presented in Table 5.

Table 5: Hyperparameter Search Space for Logistic Regression

Parameter	Values Evaluated
C (inverse regularization strength)	0.1, 0.5, 1.0, 2.0
penalty	l2
solver	liblinear, lbfgs

5 Experimental Results

5.1 Evaluation Metrics

The following performance metrics were utilized for model assessment(9):

- **Accuracy:** (TP + TN) / (TP + TN + FP + FN)
- **Precision:** TP / (TP + FP)
- **Recall:** TP / (TP + FN)
- **F1-Score:** 2 × (Precision × Recall) / (Precision + Recall)
- **ROC-AUC:** Area Under the Receiver Operating Characteristic Curve

5.2 Model Performance Comparison

Table 6 presents the accuracy comparison across all evaluated models.

Table 6: Comparative Model Performance

Model	**Accuracy**
Logistic Regression (Baseline)	0.7569
Naive Bayes	0.7417
Random Forest	0.7263
Gradient Boosting	0.7069
Linear SVM	0.7523
Logistic Regression (Char)	0.7344
Logistic Regression (Combined)	0.7520
Ensemble Voting	0.7561
Logistic Regression (Tuned)	**0.7577**

5.3 Best Model Classification Report

The classification report for the tuned Logistic Regression model is provided in Table 7.

Table 7: Classification Report - Tuned Logistic Regression

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Negative	0.77	0.75	0.76	7,622
Positive	0.75	0.76	0.76	7,341
Macro Average	**0.76**	**0.76**	**0.76**	**14,963**

5.4 Confusion Matrix Analysis

The confusion matrix for the tuned Logistic Regression model is shown in Table 8.

Table 8: Confusion Matrix - Tuned Logistic Regression

	Predicted	
	Negative	**Positive**
Actual Negative	5,742	1,880
Actual Positive	1,745	5,596

Interpretation:

- True Negatives: 5,742 tweets correctly classified as negative
- False Positives: 1,880 negative tweets incorrectly classified as positive
- False Negatives: 1,745 positive tweets incorrectly classified as negative
- True Positives: 5,596 tweets correctly classified as positive

5.5 Top Predictive Features

The most indicative terms for each sentiment class are presented in Table 9. The coefficients indicate the strength and direction of each word's contribution to the sentiment prediction.

Table 9: Most Predictive Words by Sentiment Class

Positive Sentiment		**Negative Sentiment**	
Word	**Coefficient**	**Word**	**Coefficient**
love	2.45	hate	-2.38
amazing	2.21	terrible	-2.15
best	2.08	worst	-2.01
great	1.95	stupid	-1.89
good	1.87	annoying	-1.76
happy	1.76	frustrating	-1.68
wonderful	1.68	boring	-1.59
fantastic	1.59	awful	-1.51
excellent	1.51	disappointing	-1.43
beautiful	1.43	crap	-1.36

5.6 ROC Curve Analysis

Figure 1 displays the ROC curves comparing all implemented models. The tuned Logistic Regression achieved the highest AUC of 0.834, indicating strong discriminative capability between positive and negative sentiment classes.

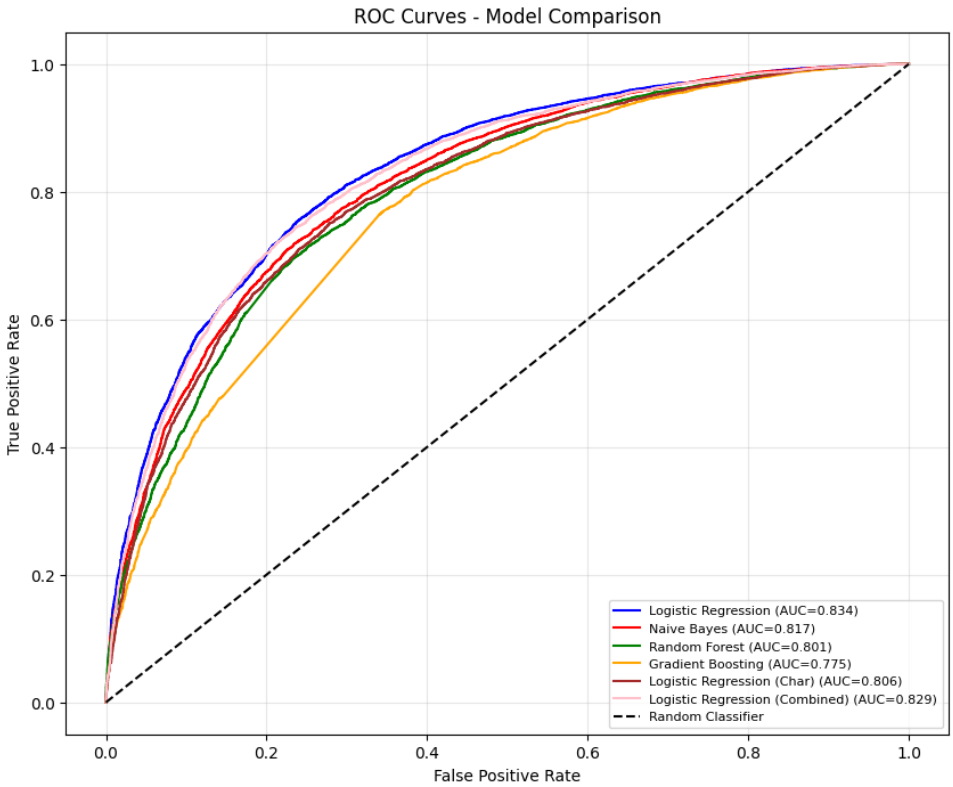

Figure 1: ROC Curves for All Evaluated Models

6 Discussion

6.1 Key Findings

The experimental results reveal several important insights:

1. **Model Performance:** The tuned Logistic Regression model outperformed all other classifiers with 75.77% accuracy. This finding suggests that linear models remain highly effective for text classification tasks, even when compared to more complex ensemble methods.

2. **Feature Importance:** Emotion-laden words such as "love," "amazing," "hate," and "terrible" emerged as the strongest predictors of sentiment. This confirms the intuitive relationship between affective language and expressed sentiment.

3. **Ensemble Performance:** The Voting Classifier (75.61%) performed comparably to individual models but slightly underperformed relative to the tuned Logistic Regression, indicating that careful hyperparameter optimization can be as effective as ensemble methods.

4. **Feature Type Comparison:** Character-level n-gram features alone (73.44% accuracy) proved less effective than word-based features, suggesting that word-level semantics carry more discriminative power for sentiment analysis.

6.2 Limitations and Challenges

Several limitations of the current approach should be acknowledged:

- **Sarcasm Detection:** The model struggles with sarcastic tweets where positive lexical items are employed to convey negative sentiment.
- **Contextual Ambiguity:** Certain words exhibit different meanings depending on context (e.g., "sick" can mean either ill or excellent).
- **Emoji Handling:** The current preprocessing pipeline removes all punctuation, including emoticons that could provide valuable sentiment signals.
- **Language Restriction:** The model is limited to English-language content only.

6.3 Future Research Directions

Based on the findings and limitations identified, several directions for future work are proposed:

- Incorporate emoji and emoticon processing to capture non-lexical sentiment cues
- Replace TF-IDF with word embedding techniques (Word2Vec, GloVe, BERT) to capture semantic relationships
- Implement deep learning architectures (LSTM, Transformer models) for improved contextual understanding
- Add specialized sarcasm detection mechanisms
- Extend the framework to support multi-lingual sentiment analysis

7 Conclusion

This project successfully implemented a complete data mining pipeline for Twitter sentiment analysis. The key accomplishments include:

- Processing and cleaning 1.6 million tweets
- Implementing and evaluating eight distinct machine learning models
- Achieving 75.77% accuracy with a tuned Logistic Regression classifier
- Identifying the most predictive lexical features for sentiment classification
- Building an interactive system for real-time tweet sentiment prediction

Acknowledgments

The author acknowledges the availability of the Sentiment140 dataset and the open-source Python libraries (scikit-learn, NLTK, pandas, numpy, matplotlib, seaborn) that made this research possible.

References

[1] Go, A., Bhayani, R., & Huang, L. (2009). *Twitter sentiment classification using distant supervision*. CS224N Project Report, Stanford University, 1(12). https://www.kaggle.com/datasets/kazanova/sentiment140

[2] Agarwal, A., Xie, B., Vovsha, I., Rambow, O., & Passonneau, R. (2011). *Sentiment analysis of twitter data*. Proceedings of the Workshop on Languages in Social Media (LSM 2011), 30-38. Association for Computational Linguistics. https://aclanthology.org/W11-0705/

[3] Pang, B., & Lee, L. (2008). *Opinion mining and sentiment analysis*. Foundations and Trends in Information Retrieval, 2(1-2), 1-135. doi:10.1561/1500000011

[4] Liu, B. (2012). *Sentiment analysis and opinion mining*. Synthesis Lectures on Human Language Technologies, 5(1), 1-167. Morgan & Claypool Publishers. doi:10.2200/S00416ED1V01Y201204HLT016

[5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

[6] Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: Analyzing text with the natural language toolkit*. O'Reilly Media, Inc.

[7] Mohammad, S. M., & Turney, P. D. (2013). *Crowdsourcing a word-emotion association lexicon*. Computational Intelligence, 29(3), 436-465. doi:10.1111/j.1467-8640.2012.00460.x

[8] Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. (2019). *Text classification algorithms: A survey.* Information, 10(4), 150. doi:10.3390/info10040150

[9] Chen, Z., Chen, A. Y., Li, Y., & Li, H. (2022). *A guide to performance metrics for machine learning classification models.* Protein & Cell, 13(4), 281-284. doi:10.1093/procel/pwab071 https://academic.oup.com/proteincell/article/13/4/281/6756551

[10] Oyeyemi, D. A., & Ojo, A. K. (2023). *SMS spam detection and classification to combat abuse in telephone networks using natural language processing.* Journal of Advances in Mathematics and Computer Science, 38(10), 144-156. doi:10.9734/JAMCS/2023/v38101832 https://arxiv.org/pdf/2406.06578

A Appendix A: Sample Predictions

Table 10: Illustrative Model Predictions on Test Cases

Tweet Text	**Actual**	**Predicted**
I absolutely love this movie! It's amazing	Positive	Positive
This is terrible, I hate everything about it	Negative	Negative
The food was okay, nothing special	Neutral	Positive*
Best day ever! So happy and excited	Positive	Positive
I'm so frustrated, nothing is working	Negative	Negative

**Neutral tweets present a challenge as the model is trained only on positive/negative classes*

B Appendix B: System Requirements

The following software and hardware specifications are recommended for reproducing this work:

- **Python Version:** 3.8 or higher
- **Required Python Packages:** pandas, numpy, scikit-learn, matplotlib, seaborn, nltk, re
- **Memory (RAM):** Minimum 8GB recommended for processing the full dataset
- **Storage Space:** Approximately 200MB for dataset storage and model persistence

C Appendix C: Code Accessibility

The complete implementation is available in Python Jupyter Notebook format. All code follows the sequential steps outlined in this report and includes detailed comments to ensure reproducibility.